Bahria University,

Karachi Campus

COURSE: CSL-210 OBJECT ORIENTED PROGRAMMING
TERM: SPRING 2024, CLASS: BSE- 2 (C)

PROJECT NAME

RFB (Royal FaceBook)

Submitted By:

S.NO	Student Name	Enrollment #
1	Mohammad Abubakar Afzal	02-131232-052
2	Raja Ubaidullah	02-131232-030
3	Fiza Khan	02-131232-048

Submitted to:

Engr.Mahawish/ Engr. Saniya Sarim

Signed **Remarks:** **Score:**

Table of Contents

INTRODUCTION 3
OOP CONCEPTS USED IN PROJECT 3
UML CLASS DIAGRAM 5
FEATURES OF OUR PROJECT 5
CODE 7
INTERFACES 21
CONCLUSION 23

INTRODUCTION

Step into the world of RFB, where real-time conversations meet stunning visuals. This revolutionary chatting application combines the power of socket programming and SQL database to deliver a seamless and engaging experience. Send and receive messages, emojis, and view chat history with ease, and discover a world of possibilities with features like profile viewing, secure Access Portal, and an AI chatbot that's always ready to assist. With RFB, the boundaries of online social interactions are pushed to new heights, making it the ultimate platform for anyone looking to connect with friends, family, and like-minded individuals. Through the use of Swing components and event listeners, the application provides a responsive and user-friendly interface. This structured approach not only enhances code readability but also facilitates future enhancements and debugging.

OOP CONCEPTS USED IN PROJECT

The RFB Chatting Application demonstrates effective use of core OOP concepts such as encapsulation, inheritance, polymorphism, and abstraction. Each class encapsulates specific functionalities, promoting modularity and maintainability. Inheritance from Swing components enables reuse of existing functionality, while polymorphism and event-driven programming provide flexibility and responsiveness.

1. Encapsulation:

Encapsulation is the bundling of data with the methods that operate on that data. In your project, encapsulation is evident in the following ways:

Class Definitions: Each class in your project (e.g., Client, ChatWindow, ProfilePage) encapsulates its data (fields) and methods. For example, the ProfilePage class encapsulates the user profile information and methods to display it.

Access Modifiers: The use of private fields and public methods ensures that the internal state of the objects is protected from unauthorized access and modification. For instance, in the ProfilePage class, details such as name, profilePic, and isOnline are passed via the constructor and encapsulated within the class.

2. Inheritance:

Inheritance is the mechanism by which one class can inherit the properties and methods of another class. In your project, inheritance is used in the following ways:

Swing Components: Your classes extend Swing components to create custom GUI elements. For example, ProfilePage extends JFrame to inherit its properties and methods, enabling it to function as a window.

3. Polymorphism:

Polymorphism allows objects to be treated as instances of their parent class rather than their actual class. This concept is demonstrated in your project as follows:

ActionListeners and MouseListeners: By implementing interfaces such as ActionListener and MouseAdapter, your classes can handle different types of events. The Client class, for example, implements ActionListener to handle button click events polymorphically.

ListCellRenderer: The ChatListRenderer class implements ListCellRenderer to customize the rendering of list items, allowing for polymorphic behavior in how each list item is displayed.

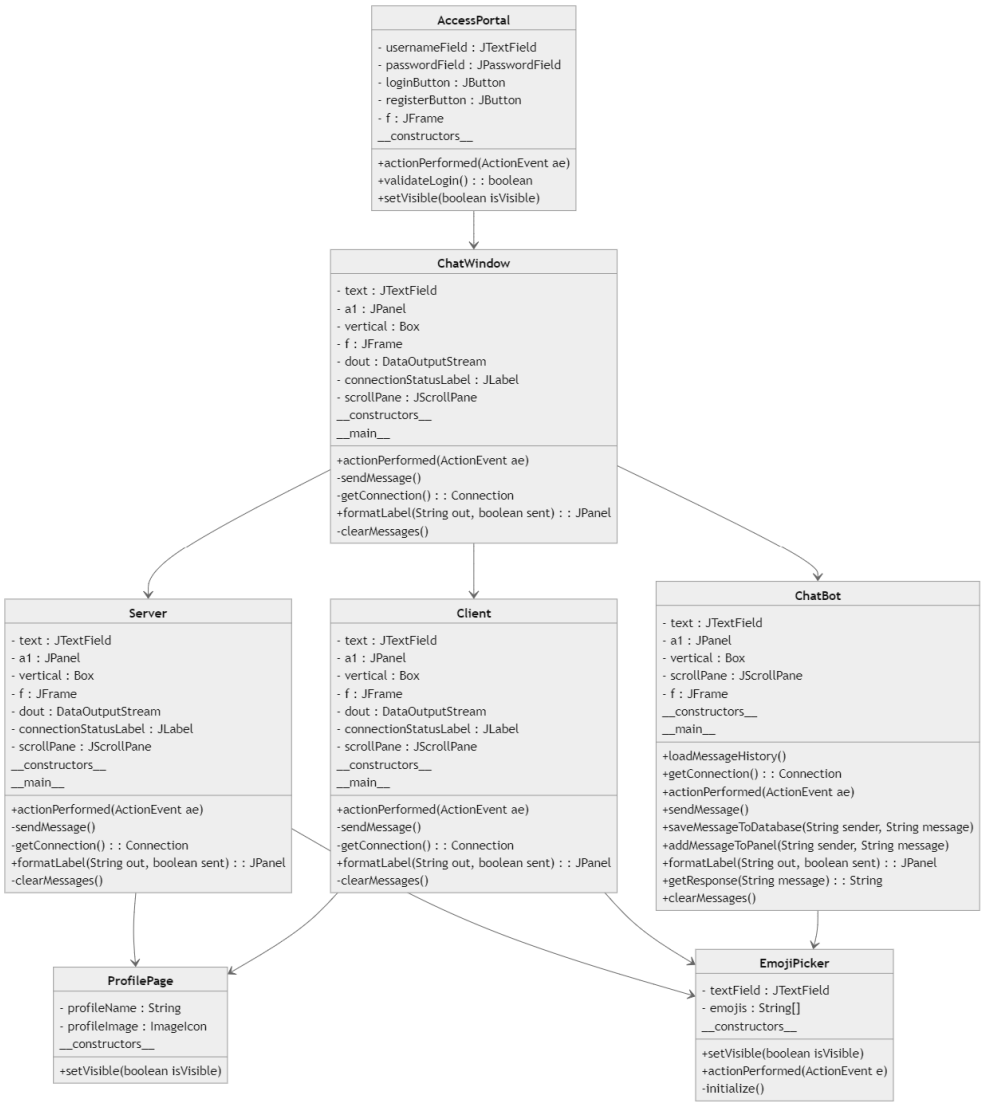
4. Abstraction:

Abstraction involves hiding complex implementation details and showing only the necessary features. In your project, abstraction is utilized as follows:

GUI Components: The use of Swing components abstracts the complex details of rendering graphical elements. For example, using JLabel, JButton, and JScrollPane abstracts the drawing and event handling details.

Networking: The handling of socket connections abstracts the low-level details of network communication. The Client class uses Socket, DataInputStream, and DataOutputStream to abstract the complexities of data transmission over a network.

UML CLASS DIAGRAM



FEATURES OF OUR PROJECT:

Following are the main features of Royal Facebook (RFB):

Graphical User Interface (GUI):

The application boasts a user-friendly GUI created using Java Swing components. This includes panels, labels, buttons, and text fields, all designed to provide a visually appealing and intuitive interface.

Chat Functionality:

Users can send and receive text messages in real-time. The messages are displayed in a chat window, with a distinction between sent and received messages for clarity.

Profile Management:

Users can view profile information through the ProfilePage class. This includes the profile picture, name, email,

phone number, and online status.

Connection Status Indicator:

The application provides feedback on the connection status with messages like "Waiting for the other user to be connected" and "Connection established successfully." This helps users understand the state of their chat connection.

Emoji Support:

An emoji button is available to open an emoji picker, allowing users to include emojis in their messages, making the chat more expressive and engaging.

Search Functionality:

The chat window includes a search bar to filter chat list items. Users can quickly find specific chats by typing keywords, enhancing the usability of the application.

Multiple Chat Support:

Users can switch between different chats. The chat list displays multiple contacts, and selecting a contact opens the corresponding chat window.

Profile Picture and Name Display:

Each chat displays the profile picture and name of the contact, making it easier for users to identify who they are chatting with.

Exit Button:

The application includes an exit button for users to close the chat application easily.

Scroll Pane:

The chat area is enclosed in a scroll pane, allowing users to scroll through past messages smoothly.

Message Formatting:

Messages are formatted with HTML to ensure they are displayed in a readable and visually appealing manner. This includes appropriate padding, font styles, and background colors for sent and received messages.

Event Handling:

The application responds to various user actions such as button clicks and key presses through event listeners, providing an interactive experience.

Placeholder Text:

Text fields for typing messages and searching include placeholder text to guide users on their intended use.

Real-time Clock:

Each message includes a timestamp, helping users keep track of when messages were sent or received.

Pop-up Notifications:

The application displays pop-up notifications to inform users when the other party disconnects or when the server goes offline

CODE:

Server Code:

```
package chatting.application;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Font;
import java.awt.Image;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.KeyAdapter;
import java.awt.event.KeyEvent;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import java.io.DataInputStream;
import java.io.DataOutputStream;
import java.net.ServerSocket;
import java.net.Socket;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.Statement;
import java.text.SimpleDateFormat;
import java.util.Calendar;
import javax.swing.*;
import javax.swing.border.EmptyBorder;


public class Server implements ActionListener {
   JTextField text;
   static JPanel a1;
   static Box vertical = Box.createVerticalBox();
   static JFrame f = new JFrame();
   static DataOutputStream dout;
   static JLabel connectionStatusLabel;
   static JScrollPane scrollPane;

   Server() {
      f.setLayout(null);
      f.setTitle("Chatme");
      f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
f.setResizable(false);
JPanel p1 = new JPanel();
p1.setBackground(Color.BLACK);
p1.setBounds(0, 0, 450, 70);
p1.setLayout(null);
f.add(p1);

connectionStatusLabel = new JLabel("<html><p style=\"width: 350px\">Waiting for the other user to be connected</p></html>");
connectionStatusLabel.setBounds(55, 75, 440, 570);
connectionStatusLabel.setFont(new Font("Helvetica", Font.PLAIN, 16));
connectionStatusLabel.setForeground(Color.RED);
f.add(connectionStatusLabel);

ImageIcon i1 = new ImageIcon(ClassLoader.getSystemResource("icons/3.png"));
Image i2 = i1.getImage().getScaledInstance(25, 25, Image.SCALE_DEFAULT);
ImageIcon i3 = new ImageIcon(i2);
JLabel back = new JLabel(i3);
back.setBounds(5, 20, 25, 25);
p1.add(back);

back.addMouseListener(new MouseAdapter() {
   public void mouseClicked(MouseEvent ae) {
      System.exit(0);
   }
});

ImageIcon i4 = new ImageIcon(ClassLoader.getSystemResource("icons/fiza bakar.png"));
Image i5 = i4.getImage().getScaledInstance(60, 60, Image.SCALE_DEFAULT);
ImageIcon i6 = new ImageIcon(i5);
JLabel profile = new JLabel(i6);
profile.setBounds(40, 10, 50, 50);
p1.add(profile);

ImageIcon i7 = new ImageIcon(ClassLoader.getSystemResource("icons/video.png"));
Image i8 = i7.getImage().getScaledInstance(30, 30, Image.SCALE_DEFAULT);
ImageIcon i9 = new ImageIcon(i8);
JLabel video = new JLabel(i9);
video.setBounds(300, 20, 30, 30);
p1.add(video);

ImageIcon i10 = new ImageIcon(ClassLoader.getSystemResource("icons/phone.png"));
Image i11 = i10.getImage().getScaledInstance(35, 30, Image.SCALE_DEFAULT);
ImageIcon i12 = new ImageIcon(i11);
JLabel phone = new JLabel(i12);
phone.setBounds(360, 20, 35, 30);
p1.add(phone);

ImageIcon i13 = new ImageIcon(ClassLoader.getSystemResource("icons/Dots.png"));
Image i14 = i13.getImage().getScaledInstance(30, 23, Image.SCALE_DEFAULT);
ImageIcon i15 = new ImageIcon(i14);
JLabel morevent = new JLabel(i15);
morevent.setBounds(420, 20, 10, 25);

morevent.addMouseListener(new MouseAdapter() {
   @Override
   public void mouseClicked(MouseEvent ae) {
```

```
            int confirm = JOptionPane.showConfirmDialog(f, "Are you sure you want to clear all messages?", "Clear Messages",
JOptionPane.YES_NO_OPTION);
            if (confirm == JOptionPane.YES_OPTION) {
                clearMessages();
            }
        }
    });

    p1.add(morevent);

    JLabel name = new JLabel("Fiza Chaudhary");
    name.setBounds(110, 15, 140, 20);
    name.setForeground(Color.WHITE);
    name.setFont(new Font("Helvetica", Font.BOLD, 16));
    p1.add(name);

    JLabel status = new JLabel("Active Now");
    status.setBounds(110, 35, 100, 18);
    status.setForeground(Color.WHITE);
    status.setFont(new Font("Helvetica", Font.PLAIN, 16));
    p1.add(status);

    a1 = new JPanel();
    a1.setBounds(5, 75, 440, 540);
    a1.setBackground(Color.BLACK);
    f.add(a1);

    text = new JTextField();
    text.setBounds(1, 620, 330, 40);
    text.setFont(new Font("Helvetica", Font.PLAIN, 16));
    text.setText("Type a message");

    MouseAdapter profileClickListener = new MouseAdapter() {
        @Override
        public void mouseClicked(MouseEvent e) {
            ProfilePage profilePage = new ProfilePage("Fiza Chaudhary", i6, true);
            profilePage.setVisible(true);
        }
    };
    profile.addMouseListener(profileClickListener);
    name.addMouseListener(profileClickListener);

    text.addKeyListener(new KeyAdapter() {
        @Override
        public void keyReleased(KeyEvent e) {
            if (text.getText().isEmpty()) {
                text.setText("Type a message");
            }
        }

        public void keyTyped(KeyEvent e) {
            if (text.getText().equals("Type a message")) {
                text.setText("");
            }
        }
    });
```

```
    f.add(text);

    JButton send = new JButton("Send");
    send.setBounds(330, 620, 70, 40);
    send.setBackground(Color.BLACK);
    send.setForeground(Color.WHITE);
    send.addActionListener(this);
    send.setFont(new Font("Helvetica", Font.PLAIN, 14));
    f.add(send);

    JButton emojiButton = new JButton("☺");
    emojiButton.setBounds(400, 620, 50, 40);
    emojiButton.setBackground(Color.BLACK);
    emojiButton.setForeground(Color.WHITE);
    emojiButton.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            EmojiPicker emojiPicker = new EmojiPicker(text);
            emojiPicker.setVisible(true);
        }
    });
    f.add(emojiButton);

    scrollPane = new JScrollPane(a1);
    scrollPane.setBounds(5, 75, 450, 540);
    scrollPane.setVerticalScrollBarPolicy(ScrollPaneConstants.VERTICAL_SCROLLBAR_ALWAYS);
    scrollPane.getVerticalScrollBar().addAdjustmentListener(e -> {
        e.getAdjustable().setValue(e.getAdjustable().getMaximum());
    });

    f.add(scrollPane);

    f.setSize(470, 700);
    f.setLocation(200, 20);
    f.getContentPane().setBackground(Color.BLACK);

    f.setVisible(true);
}

public void actionPerformed(ActionEvent ae) {
    sendMessage();
}

private void sendMessage() {
    try {
        String out = text.getText();
        JPanel p2 = formatLabel(out, true);
        a1.setLayout(new BorderLayout());
        JPanel right = new JPanel(new BorderLayout());
        right.setBackground(Color.BLACK);
        right.add(p2, BorderLayout.LINE_END);
        vertical.add(right);
        vertical.add(Box.createVerticalStrut(15));
        a1.add(vertical, BorderLayout.PAGE_START);
        dout.writeUTF(out);
```

```
        text.setText("");
        f.repaint();
        f.revalidate();

        try (Connection con = getConnection()) {
            String query = "INSERT INTO messages (sender, message) VALUES (?, ?)";
            PreparedStatement ps = con.prepareStatement(query);
            ps.setString(1, "Server");
            ps.setString(2, out);
            ps.executeUpdate();
        }

        scrollPane.getVerticalScrollBar().setValue(scrollPane.getVerticalScrollBar().getMaximum());
    } catch (Exception e) {
        e.printStackTrace();
    }
}

private Connection getConnection() {
    Connection con = null;
    try {
        String url = "jdbc:mysql://localhost:3306/chat_history";
        String user = "root";
        String password = "Chatapp@bse2c";
        Class.forName("com.mysql.cj.jdbc.Driver");
        con = DriverManager.getConnection(url, user, password);
        System.out.println("Connection established successfully.");
    } catch (Exception e) {
        e.printStackTrace();
    }
    return con;
}


public static JPanel formatLabel(String out, boolean sent) {
JPanel panel = new JPanel();
panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));

JLabel output = new JLabel("<html><p style=\"width: 150px\">" + out + "</p></html>");
output.setFont(new Font("Helvetica", Font.PLAIN, 16));
output.setOpaque(true);
output.setBorder(new EmptyBorder(15, 15, 15, 50));

if (sent) {
    output.setBackground(new Color(0, 150, 230));
    panel.setBackground(Color.BLACK);
} else {
    output.setBackground(Color.WHITE);
    panel.setBackground(Color.BLACK);
}

panel.add(output);

Calendar cal = Calendar.getInstance();
SimpleDateFormat sdf = new SimpleDateFormat("HH:mm");
```

```
    JLabel time = new JLabel();
    time.setText(sdf.format(cal.getTime()));

    panel.add(time);

    return panel;
}


    private void clearMessages() {
        a1.removeAll();
        vertical.removeAll();
        a1.revalidate();
        a1.repaint();
        f.revalidate();
        f.repaint();

        try (Connection con = getConnection()) {
            String query = "DELETE FROM messages";
            Statement stmt = con.createStatement();
            stmt.executeUpdate(query);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

   public static void main(String[] args) {
        Server server = new Server();
        Connection testCon = server.getConnection();
        if (testCon != null) {
            System.out.println("Database connection successful!");
        } else {
            System.out.println("Failed to establish database connection.");
        }

        try {
            ServerSocket skt = new ServerSocket(6666);
            while (true) {
                Socket s = skt.accept();
                DataInputStream din = new DataInputStream(s.getInputStream());
                dout = new DataOutputStream(s.getOutputStream());
                connectionStatusLabel.setText("<html><p style=\"width: 350px\">Connection established successfully.<br>Now you can
chat.</p></html>");
                connectionStatusLabel.setForeground(new Color(0x44C826));
                Timer timer = new Timer(3000, new ActionListener() {
                    @Override
                    public void actionPerformed(ActionEvent e) {
                        connectionStatusLabel.setVisible(false);
                        f.getContentPane().remove(connectionStatusLabel);
                        f.revalidate();
                    }
                });
                timer.setRepeats(false);
                timer.start();

                try (Connection con = server.getConnection()) {
```

```
    if (con != null) {
        String query = "SELECT sender, message FROM messages";
        Statement stmt = con.createStatement();
        ResultSet rs = stmt.executeQuery(query);
        while (rs.next()) {
            String sender = rs.getString("sender");
            String message = rs.getString("message");
            JPanel panel = null;
            if ("Server".equals(sender)) {
                panel = formatLabel(message, true);
            } else if ("Client".equals(sender)){
                panel = formatLabel(message, false);
            }
            JPanel right = new JPanel(new BorderLayout());
            JPanel left = new JPanel(new BorderLayout());
            right.setBackground(Color.BLACK);
            left.setBackground(Color.BLACK);
            if ("Server".equals(sender)) {
                right.add(panel, BorderLayout.LINE_END);
                vertical.add(right);
            } else if ("Client".equals(sender)){
                left.add(panel, BorderLayout.LINE_START);
                vertical.add(left);
            }
        }

    }
} catch (Exception e) {
    e.printStackTrace();
}

while (true) {
    String msg;
    try {
        msg = din.readUTF();
        if (msg.equals("exit")) {
            JOptionPane.showMessageDialog(f, "Client has disconnected.");
            skt.close();
            break;
        }
    } catch (Exception e) {
        JOptionPane.showMessageDialog(f, "Client has disconnected.");
        skt.close();
        break;
    }
    JPanel panel = server.formatLabel(msg, false);
    JPanel left = new JPanel(new BorderLayout());
    left.setBackground(Color.BLACK);
    left.add(panel, BorderLayout.LINE_START);
    vertical.add(left);

    try (Connection con = server.getConnection()) {
        if (con != null) {
            String query = "INSERT INTO messages (sender, message) VALUES (?, ?)";
            PreparedStatement ps = con.prepareStatement(query);
            ps.setString(1, "Client");
```

```
                ps.setString(2, msg);
                ps.executeUpdate();
            }
        } catch (Exception e) {
            e.printStackTrace();
        }

        scrollPane.getVerticalScrollBar().setValue(scrollPane.getVerticalScrollBar().getMaximum());
        f.validate();
        }
    }
  } catch (Exception e) {
    e.printStackTrace();
  }
 }
}
```

Client Code:

```
package chatting.application;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Font;
import java.awt.Image;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.KeyAdapter;
import java.awt.event.KeyEvent;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import java.io.DataInputStream;
import java.io.DataOutputStream;
import java.net.Socket;
import java.text.SimpleDateFormat;
import java.util.Calendar;
import javax.swing.Box;
import javax.swing.BoxLayout;
import javax.swing.ImageIcon;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextField;
import javax.swing.ScrollPaneConstants;
import javax.swing.Timer;
import javax.swing.border.EmptyBorder;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.Statement;
```

```
public class Client implements ActionListener {
    JTextField text;
    static JPanel a1;
    static Box vertical = Box.createVerticalBox();
    static JScrollPane scrollPane;
    static JFrame f = new JFrame();
    static DataOutputStream dout;
    static JLabel connectionStatusLabel;

    Client() {
        f.setLayout(null);
        f.setTitle("RFB");
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        JPanel p1 = new JPanel();
        p1.setBackground(Color.BLACK);
        p1.setBounds(0, 0, 450, 70);
        p1.setLayout(null);
        f.add(p1);

        connectionStatusLabel = new JLabel("<html><p style=\"width: 350px\">"
                + "Waiting for the other user to be connected" + "</p></html>");
        connectionStatusLabel.setBounds(55, 75, 440, 570);
        connectionStatusLabel.setFont(new Font("Helvetica", Font.PLAIN, 16));
        connectionStatusLabel.setForeground(Color.RED);
        f.add(connectionStatusLabel);

        ImageIcon i1 = new ImageIcon(ClassLoader.getSystemResource("icons/3.png"));
        Image i2 = i1.getImage().getScaledInstance(25, 25, Image.SCALE_DEFAULT);
        ImageIcon i3 = new ImageIcon(i2);
        JLabel back = new JLabel(i3);
        back.setBounds(5, 20, 25, 25);
        p1.add(back);

        back.addMouseListener(new MouseAdapter() {
            public void mouseClicked(MouseEvent ae) {
                System.exit(0);
            }
        });

        ImageIcon i4 = new ImageIcon(ClassLoader.getSystemResource("icons/raja.jpeg"));
        Image i5 = i4.getImage().getScaledInstance(50, 50, Image.SCALE_DEFAULT);
        ImageIcon i6 = new ImageIcon(i5);
        JLabel profile = new JLabel(i6);
        profile.setBounds(40, 10, 50, 50);
        p1.add(profile);

        ImageIcon i7 = new ImageIcon(ClassLoader.getSystemResource("icons/video.png"));
        Image i8 = i7.getImage().getScaledInstance(30, 30, Image.SCALE_DEFAULT);
        ImageIcon i9 = new ImageIcon(i8);
        JLabel video = new JLabel(i9);
        video.setBounds(300, 20, 30, 30);
        p1.add(video);

        ImageIcon i10 = new ImageIcon(ClassLoader.getSystemResource("icons/phone.png"));
        Image i11 = i10.getImage().getScaledInstance(35, 30, Image.SCALE_DEFAULT);
        ImageIcon i12 = new ImageIcon(i11);
```

```
        JLabel phone = new JLabel(i12);
        phone.setBounds(360, 20, 35, 30);
        p1.add(phone);

        ImageIcon i13 = new ImageIcon(ClassLoader.getSystemResource("icons/Dots.png"));
        Image i14 = i13.getImage().getScaledInstance(30, 23, Image.SCALE_DEFAULT);
        ImageIcon i15 = new ImageIcon(i14);
        JLabel morevent = new JLabel(i15);
        morevent.setBounds(420, 20, 10, 25);
         morevent.addMouseListener(new MouseAdapter() {
            @Override
            public void mouseClicked(MouseEvent ae) {
                int confirm = JOptionPane.showConfirmDialog(f, "Are you sure you want to clear all messages?", "Clear Messages",
JOptionPane.YES_NO_OPTION);
                if (confirm == JOptionPane.YES_OPTION) {
                    clearMessages();
                }
            }
        });

        p1.add(morevent);

        JLabel name = new JLabel("Raja");
        name.setBounds(110, 15, 100, 18);
        name.setForeground(Color.WHITE);
        name.setFont(new Font("Helvetica", Font.BOLD, 16));
        p1.add(name);

        JLabel status = new JLabel("Active Now");
        status.setBounds(110, 35, 100, 18);
        status.setForeground(Color.WHITE);
        status.setFont(new Font("Helvetica", Font.PLAIN, 16));
        p1.add(status);

        a1 = new JPanel();
        a1.setBounds(5, 75, 440, 540);
        a1.setBackground(Color.BLACK);
        f.add(a1);

        text = new JTextField();
        text.setBounds(1, 620, 330, 40);
        text.setFont(new Font("Helvetica", Font.PLAIN, 16));
        text.setText("Type a message");

        text.addKeyListener(new KeyAdapter() {
            @Override
            public void keyReleased(KeyEvent e) {
                if (text.getText().isEmpty()) {
                    text.setText("Type a message");
                }
            }

            public void keyTyped(KeyEvent e) {
                if (text.getText().equals("Type a message")) {
                    text.setText("");
                }
```

```
        }
    });

    f.add(text);

    JButton send = new JButton("Send");
    send.setBounds(330, 620, 70, 40);
    send.setBackground(Color.BLACK);
    send.setForeground(Color.WHITE);
    send.addActionListener(this);
    send.setFont(new Font("Helvetica", Font.PLAIN, 14));
    f.add(send);

    JButton emojiButton = new JButton("☺");
    emojiButton.setBounds(400, 620, 50, 40);
    emojiButton.setBackground(Color.BLACK);
    emojiButton.setForeground(Color.WHITE);
    emojiButton.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            EmojiPicker emojiPicker = new EmojiPicker(text);
            emojiPicker.setVisible(true);
        }
    });
    f.add(emojiButton);

    scrollPane = new JScrollPane(a1);
    scrollPane.setBounds(5, 75, 450, 540);
    scrollPane.setVerticalScrollBarPolicy(ScrollPaneConstants.VERTICAL_SCROLLBAR_ALWAYS);
    scrollPane.getVerticalScrollBar().addAdjustmentListener(e -> {
        e.getAdjustable().setValue(e.getAdjustable().getMaximum());
    });

    f.add(scrollPane);

    MouseAdapter profileClickListener = new MouseAdapter() {
        @Override
        public void mouseClicked(MouseEvent e) {
            ProfilePage profilePage = new ProfilePage("Raja", i6, true);
            profilePage.setVisible(true);
        }
    };
    profile.addMouseListener(profileClickListener);
    name.addMouseListener(profileClickListener);

    f.setSize(470, 700);
    f.setLocation(800, 20);
    f.getContentPane().setBackground(Color.BLACK);

    f.setVisible(true);
}

public void actionPerformed(ActionEvent ae) {
    if (ae.getActionCommand().equals("Send")) {
        sendMessage();
```

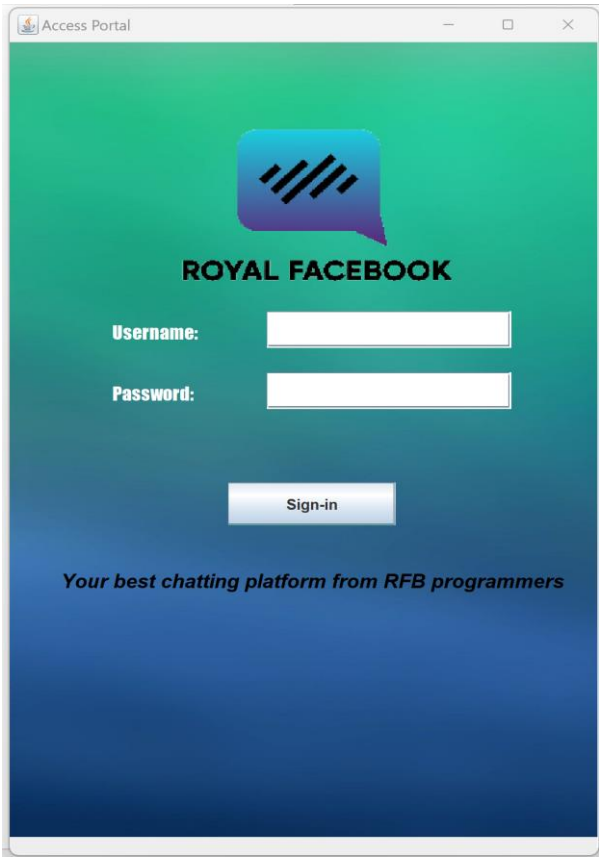
```
    }
}

private void sendMessage() {
    try {
        String out = text.getText();
        JPanel p2 = formatLabel(out, true);
        a1.setLayout(new BorderLayout());
        JPanel right = new JPanel(new BorderLayout());
        right.setBackground(Color.BLACK);
        right.add(p2, BorderLayout.LINE_END);
        vertical.add(right);
        vertical.add(Box.createVerticalStrut(15));
        a1.add(vertical, BorderLayout.PAGE_START);
        dout.writeUTF(out);
        text.setText("");
        f.repaint();
        f.revalidate();
        scrollPane.getVerticalScrollBar().setValue(scrollPane.getVerticalScrollBar().getMaximum());
    } catch (Exception e) {
        e.printStackTrace();
    }
}

private static Connection getConnection() {
    Connection con = null;
    try {
        String url = "jdbc:mysql://localhost:3306/chat_history";
        String user = "root";
        String password = "Chatapp@bse2c";
        Class.forName("com.mysql.cj.jdbc.Driver");
        con = DriverManager.getConnection(url, user, password);
        System.out.println("Connection established successfully.");
    } catch (Exception e) {
        e.printStackTrace();
    }
    return con;
}

public static JPanel formatLabel(String out, boolean sent) {
    JPanel panel = new JPanel();
    panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));

    JLabel output = new JLabel("<html><p style=\"width: 150px\">" + out + "</p></html>");
    output.setFont(new Font("Helvetica", Font.PLAIN, 16));
    output.setOpaque(true);
    output.setBorder(new EmptyBorder(15, 15, 15, 50));

    if (sent) {
        output.setBackground(new Color(0, 150, 230));
        panel.setBackground(Color.BLACK);
    } else {
        output.setBackground(Color.WHITE);
        panel.setBackground(Color.BLACK);
    }
```

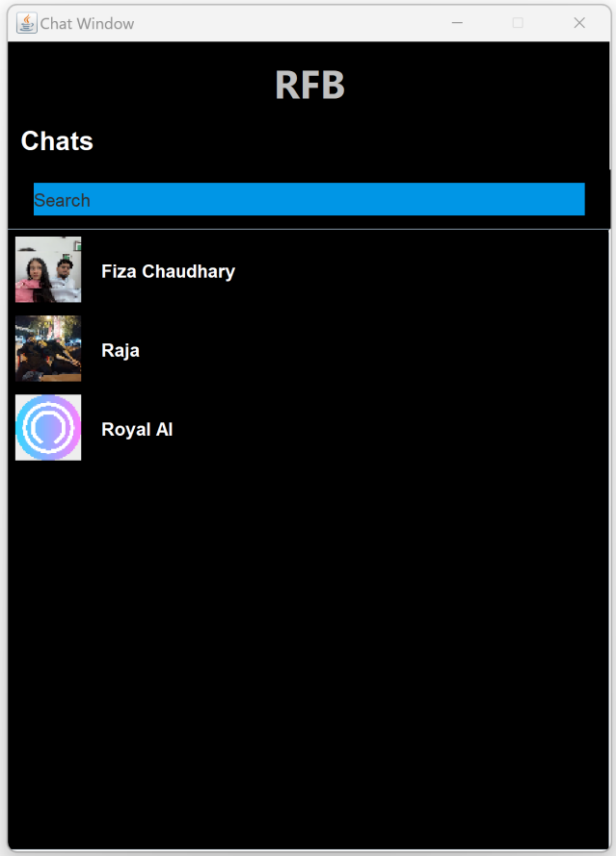
```
    panel.add(output);

    Calendar cal = Calendar.getInstance();
    SimpleDateFormat sdf = new SimpleDateFormat("HH:mm");

    JLabel time = new JLabel();
    time.setText(sdf.format(cal.getTime()));

    panel.add(time);

    return panel;
}

 private void clearMessages() {
    a1.removeAll();
    vertical.removeAll();
    a1.revalidate();
    a1.repaint();
    f.revalidate();
    f.repaint();

    try (Connection con = getConnection()) {
        String query = "DELETE FROM messages";
        Statement stmt = con.createStatement();
        stmt.executeUpdate(query);
    } catch (Exception e) {
        e.printStackTrace();
    }
}

 public static void main(String[] args) {
    Client client=new Client();
    Connection testCon = client.getConnection();
    if (testCon != null) {
        System.out.println("Database connection successful!");
    } else {
        System.out.println("Failed to establish database connection.");
    }

    try {
        Socket s = new Socket("127.0.0.101", 6666);
        DataInputStream din = new DataInputStream(s.getInputStream());
        dout = new DataOutputStream(s.getOutputStream());


        Client.connectionStatusLabel.setText("<html><p style=\"width: 350px\">"
                + "Connection established successfully." + "<br>" + "Now you can chat." + "</p></html>");
        connectionStatusLabel.setForeground(new Color(0x44C826));
        Timer timer = new Timer(3000, new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                connectionStatusLabel.setVisible(false);
                f.getContentPane().remove(connectionStatusLabel);
                f.revalidate();
            }
        });
```

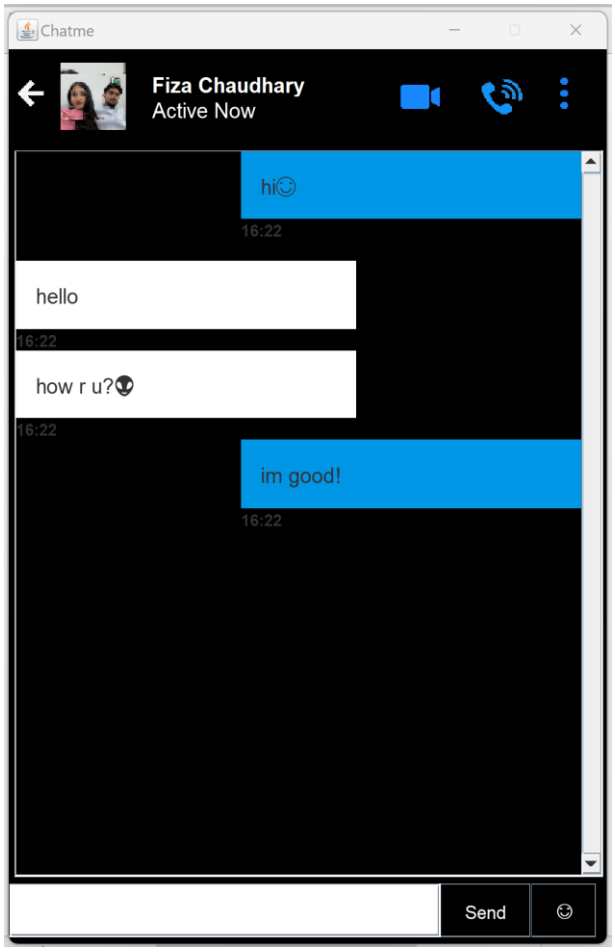
```
timer.setRepeats(false);
timer.start();

    try (Connection con = client.getConnection()) {
        if (con != null) {
            String query = "SELECT sender, message FROM messages";
            Statement stmt = con.createStatement();
            ResultSet rs = stmt.executeQuery(query);
            while (rs.next()) {
                String sender = rs.getString("sender");
                String message = rs.getString("message");
                JPanel panel = null;
                if ("Client".equals(sender)) {
                    panel = formatLabel(message, true);
                } else if("Server".equals(sender)){
                    panel = formatLabel(message, false);
                }
                JPanel right = new JPanel(new BorderLayout());
                JPanel left = new JPanel(new BorderLayout());
                right.setBackground(Color.BLACK);
                left.setBackground(Color.BLACK);
                if ("Client".equals(sender)) {
                    right.add(panel, BorderLayout.LINE_END);
                    vertical.add(right);
                } else if("Server".equals(sender)) {
                    left.add(panel, BorderLayout.LINE_START);
                    vertical.add(left);
                }
            }

        }
    } catch (Exception e) {
        e.printStackTrace();
    }

while (true) {
    a1.setLayout(new BorderLayout());
    String msg;
    try {
        msg = din.readUTF();
        if (msg.equals("exit")) {
            JOptionPane.showMessageDialog(f, "Server has disconnected.");
            s.close();
            break;
        }
    } catch (Exception e) {
        JOptionPane.showMessageDialog(f, "Server has disconnected.");
        s.close();
        break;
    }
    JPanel panel = formatLabel(msg, false);
    JPanel left = new JPanel(new BorderLayout());
    left.setBackground(Color.BLACK);
    left.add(panel, BorderLayout.LINE_START);
    vertical.add(left);
```

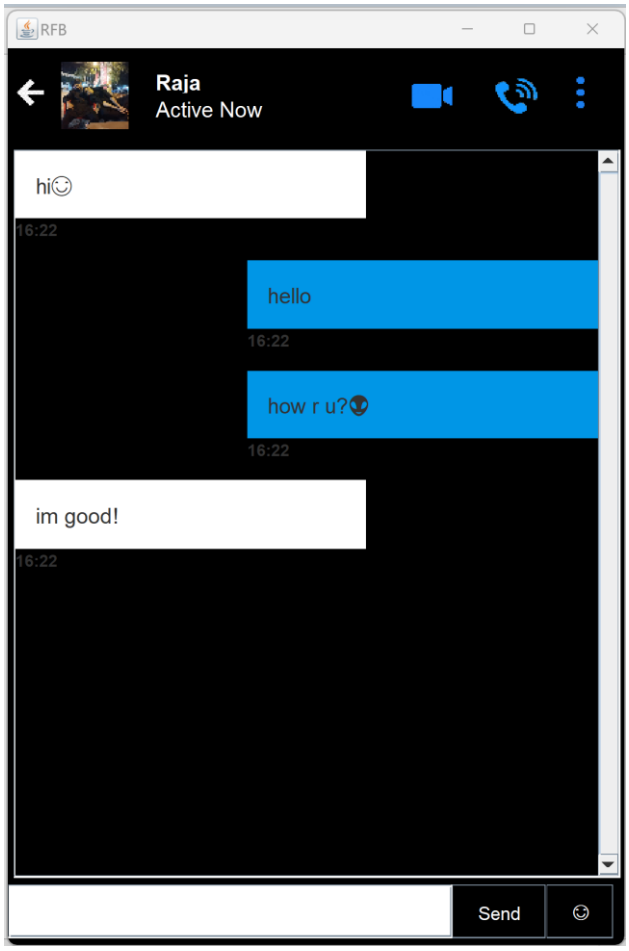
```
            try (Connection con = client.getConnection()) {
                if (con != null) {
                    String query = "INSERT INTO messages (sender, message) VALUES (?, ?)";
                    PreparedStatement ps = con.prepareStatement(query);
                    ps.setString(1, "Server");
                    ps.setString(2, msg);
                    ps.executeUpdate();
                }
            } catch (Exception e) {
                e.printStackTrace();
            }

            vertical.add(Box.createVerticalStrut(15));
            a1.add(vertical, BorderLayout.PAGE_START);
            scrollPane.getVerticalScrollBar().setValue(scrollPane.getVerticalScrollBar().getMaximum());
            f.validate();
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
  }
}
```

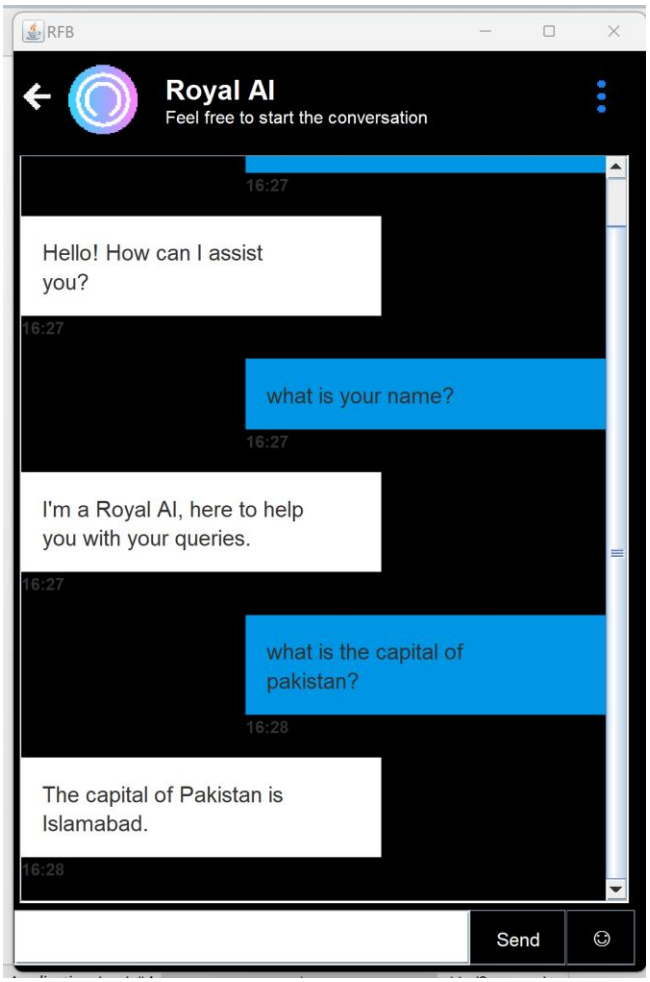
INTERFACES:

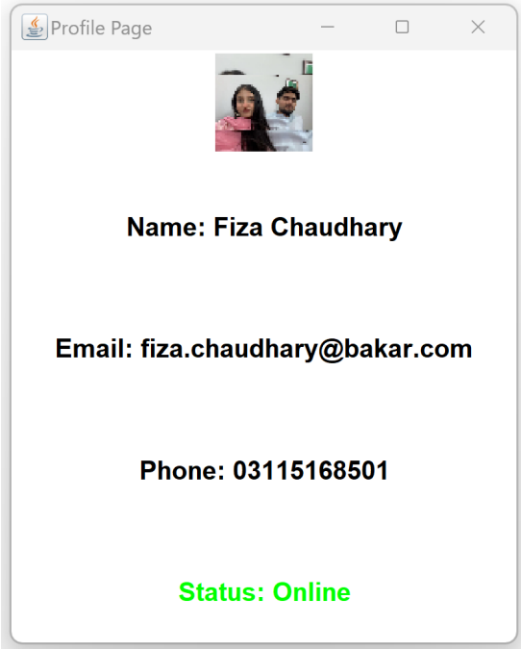


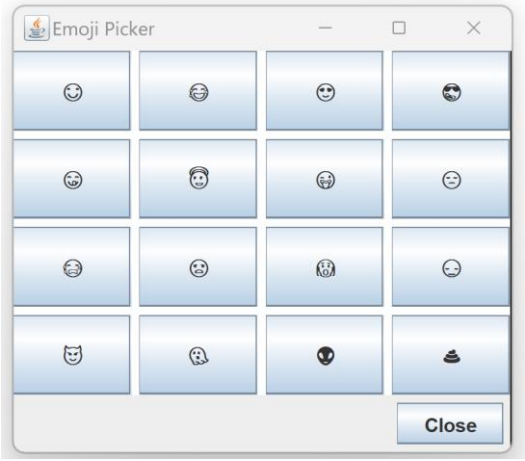



Chatme
Fiza Chaudhary
Active Now
hi☺
16:22
hello
16:22
how r u?👽
16:22
im good!
16:22
Send
☺


RFB
Raja
Active Now
hi☺
16:22
hello
16:22
how r u?👽
16:22
im good!
16:22
Send
☺







CONCLUSION:

The RFB Chatting Application project is a testament to the effective application of Object-Oriented Programming principles to create a robust, user-friendly communication platform. By utilizing Java Swing for an intuitive graphical user interface and integrating features such as real-time messaging, profile management, and emoji support, the application offers a comprehensive and engaging user experience. The well-structured class architecture ensures maintainability and scalability, while features like connection status indicators and real-time updates enhance usability and efficiency. This project not only highlights technical prowess but also demonstrates the importance of thoughtful design and user-centric features, providing a reliable and enjoyable platform for digital communication.